# Microcredit Management Application

## Project Overview and Objectives

- Purpose and Scope
  - Define project scope: Build a microcredit management app covering clients, agents, loan requests, repayments, dashboards and reporting for operational use.
  - Primary objectives: Automate loan request/granting, monitor repayments, enable risk/performance reporting, implement role-based auth (ADMIN, AGENT, CLIENT).
  - Success criteria: Reduced manual processing time, timely repayment alerts, accurate dashboards and secure role separation.
- Stakeholders and Roles
  - Stakeholder list: ADMIN (system management), AGENT (loan processing, client support), CLIENT (loan application, repayment).
  - External stakeholders: Regulators, auditors and financial partners requiring reports and compliance evidence.
  - Communication plan: Weekly sprint demos, monthly stakeholder reports, escalation channel for production incidents.
- Constraints and Assumptions
  - Technology constraints: Use Spring Boot backend, MySQL/PostgreSQL database, React/Angular/Thymeleaf frontend, Swagger for docs.
  - Security assumptions: TLS in transit, Spring Security for auth, role-based access control implemented.
  - Deployment assumptions: Cloud-hosted environment with backup and monitoring services available.

## Functional Requirements and Features

- Client and Agent Management
  - CRUD operations for clients: Create, read, update, delete client profiles with validation and audit trails.
  - Agent management: Create agent accounts, assign territories, track agent performance metrics.
  - Role assignments: Assign roles and permissions with restrictions per role for data visibility and actions.
- Loan Request and Calculation
  - Loan request workflow: Submit application, agent review, approval/denial, and fund disbursement steps defined.
  - Interest and schedule calculation: Compute amortization schedules, interest, principal breakdown and total payable amounts.
  - Eligibility checks: Automatic checks (credit history, income, existing loans) and configurable approval rules.
- Repayment Tracking and Notifications
  - Payment tracking: Record payments, allocate to schedule, update outstanding balances and delinquency status.
  - Notification system: Pre-due reminders, overdue alerts, escalation notifications via email/SMS/push.
  - Rescheduling and partial payments: Support early repayments, renegotiated schedules and partial payment processing.

## Reporting, Dashboards and Exports

- Dashboards and KPIs
  - Operational dashboards: Visuals for total granted, total repaid, outstanding principal, and delinquent loans.
  - Risk and performance KPIs: NPL ratio, average loan size, portfolio at risk, agent performance charts.
  - Real-time updates: Dashboard data refreshed on schedule or via event-driven updates for near real-time metrics.
- Report Generation and Exports
  - Export formats: PDF and CSV exports for regulatory reports, management summaries and detailed ledgers.
  - Scheduled reports: Automated daily/weekly/monthly report generation and distribution to stakeholders.
  - Audit and history reports: Transaction logs, approval history and change tracking for audit purposes.
- API and Documentation
  - OpenAPI documentation: Swagger-generated API docs covering endpoints, models and authentication flows.
  - Integration endpoints: APIs for external systems (payment gateways, SMS/email providers, accounting systems).
  - Versioning and deprecation policy: Semantic API versioning and clear deprecation notices for clients.

## Technical Architecture and Implementation

- Backend and Data Layer
  - Backend stack: Spring Boot with Spring Data JPA for persistence and Spring Security for auth and roles.
  - Database design: Normalize tables for clients, agents, loans, payments, schedules and audit logs; use MySQL or PostgreSQL.
  - Transaction management: ACID transactions for disbursement and repayment operations with optimistic locking.
- Frontend and UX
  - Frontend choices: React or Angular for SPA; Thymeleaf option for server-rendered pages if preferred.
  - Key screens: Client management, loan application form, repayment entry, agent dashboard and admin settings.
  - Accessibility and responsiveness: Mobile-first responsive design with accessibility considerations (WCAG basics).
- Security, Testing and Deployment
  - Authentication and roles: JWT or session-based auth, role checks on backend endpoints and frontend routing guards.
  - Testing strategy: Unit tests, integration tests, end-to-end tests (Cypress or Selenium) and security scans.
  - CI/CD and deployment: Automated pipelines for build/test/deploy, containerization with Docker, deploy to cloud (Kubernetes or managed services).

## Implementation Plan and Timeline

- Phased Delivery and Milestones
  - Phase 1 (MVP - 6–8 weeks): Core CRUD, loan request flow, basic repayment tracking, simple dashboard, auth/roles.
  - Phase 2 (8–10 weeks): Advanced schedule calculations, notifications, exports and extended dashboards.
  - Phase 3 (4–6 weeks): Integrations, reporting automation, scaling, hardening and production rollout.
- Action Items and Resources
  - Team composition: Backend devs (2), frontend dev (1–2), QA (1), DevOps (1), Product owner and BA.
  - Resources and tools: Git repo, CI server, issue tracker, staging environment, SMS/email provider accounts.
  - Metrics and acceptance: Uptime SLA, API response targets, test coverage thresholds and stakeholder acceptance criteria.
- Risk Management and Next Steps
  - Key risks: Data integrity during concurrent payments, regulatory compliance, notification delivery reliability.
  - Mitigation actions: Strong transactional controls, legal review, redundant notification providers and monitoring.
  - Next steps: Approve scope, allocate team, create backlog, schedule sprint 0 to finalize architecture and data model.